SECTION 1 | LECTURE 4

Spring AI · Observability · Langfuse 스택 큰 그림 보기

"잘 되느냐"보다 "어떻게 흘렀느냐"를 아키텍처 관점에서 추적하고 읽는 구조.
답변 생성 자체가 아닌 AI 흐름을 투명하게 통제하고 관측 가능한 기반 스택을 정의합니다.

APP BASE	OBSERVABILITY	LLM PANEL	HEALTH / METRICS
Spring AI	OTEL / Micrometer	Langfuse	Actuator

왜 스택을 먼저 봐야 할까

흩어진 파편적 기술 이해

단순 도구 리스트 암기

Spring AI, OTEL, Langfuse를 개별 라이브러리 단위로 따로 분석하는 관점

- **Spring AI:** ChatClient API 연동용 단독 활용
- **Langfuse:** LLM 로그 모니터링 전용 별도 웹 툴
- **OTEL / Micrometer:** 사내 표준 인프라 메트릭 수집 스택

⚠ 한계: 연결이 유실되어 AI 전체 요청의 인과관계를 디버깅하지 못함

통합 아키텍처 관점

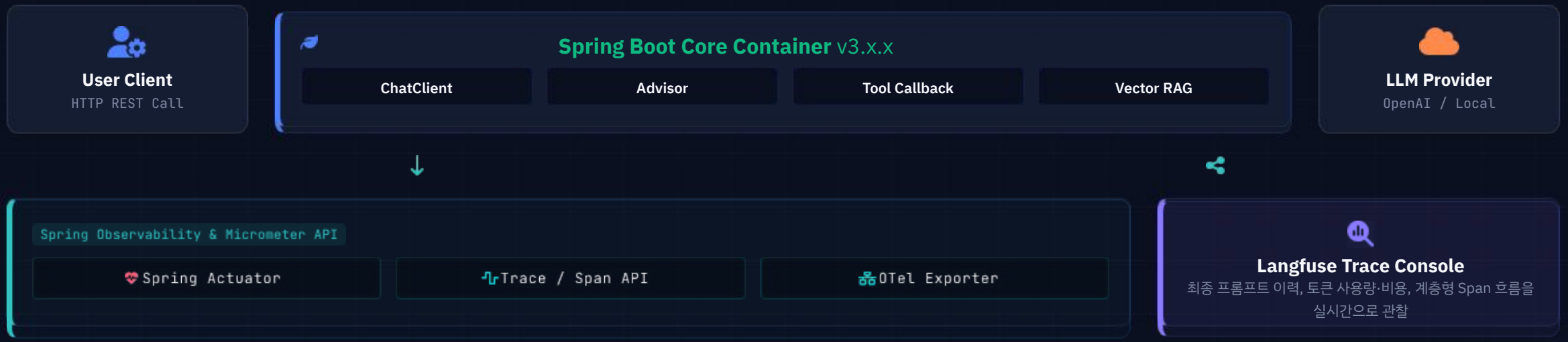
흐름 중심의 결합 설계

요청이 입력 계층부터 외부 호출, 관찰까지 매끄럽게 연결되는 통합 관측 지도

- **User Request Entry:** 사용자가 전달한 메시지의 처리 흐름 추적
- **In-Flight Application:** 컨텍스트 수립 및 분기 경로 완전 파악
- **Telemetry Export:** Trace와 Span을 통합해 지표와 흔적을 실시간 출력

목표: "요청이 언제, 어느 도구를 타고, 어떤 비용과 품질로 통과했는가"를 규명

Part 1 기술 스택 큰 그림



각 컴포넌트는 무슨 역할을 하나

01. LLM CORE

ChatClient

사용자 요청을 LLM 공급자 호출 구조로 감싸 전달하는 프레임워크 표준 입구 인터페이스

Basic Call Entry

02. CONTEXT FLOW

Advisor & RAG

호출 전후에 비즈니스 정책이나 외부 저장소 문맥(Vector) 데이터를 주입하고 필터링하는 확장 축

Context Modifier

03. STATE OBSERVATION

OTEL & Actuator

시스템 내부의 활성 커넥션, 스택의 Trace 및 Span 흔적을 안전하고 투명하게 밖으로 수집 및 가공

Telemetry Engine

04. ANALYTICS PORTAL

Langfuse Web

전달한 최종 프롬프트 정보, 누적 소모 토큰 비율, 하위 단계 실행 시간을 브라우저 대시보드로 시각화

Observability UI Portal

</> 최소 Spring AI 프로젝트의 시작점

MINIMAL IN-OUT PATH

🔍 가장 작은 단위의 호출 경로

거대한 AI 오케스트레이터를 구축하기 이전에, 사용자의 메시지가 유실되지 않고 ChatClient를 경유해 LLM으로 안전하게 왕복 배송되는 뼈대 구조를 수립합니다.



[1] ChatClient Bean Configuration

```java
@Bean
ChatClient chatClient(ChatClient.Builder builder) {
    return builder.build();
}
```

[2] ChatController Endpoint Map

```java
@RestController
@RequestMapping("/api/chat")
public class ChatController {
    private final ChatClient chatClient;

    public ChatController(ChatClient chatClient) {
        this.chatClient = chatClient;
    }

    @GetMapping
    public String chat(@RequestParam String message) {
        return this.chatClient.prompt().user(message).call().content();
    }
}
```

잘 되느냐보다 어떻게 흘렀느냐

OUTPUT-ONLY CHECK (결과 중심 관점)

단순 응답 성공 여부 확인

답변이 잘 쓰였는지와 최종 HTTP 200 성공 코드 응답만을 확인하는 운영 단계

- **블랙박스 상태:** API 호출 내부의 RAG 문서 누락 유무 파악 불가
- **잠재 비용 누수:** 한 번의 질문에서 유발된 이면의 LLM 반복 횟수 유실
- **디버깅 장벽:** 가설 생성 단계 중 어떤 prompt가 오작동했는지 분석 전무

결과: "답은 나왔지만, 왜 나왔는지 누구도 증명하지 못함"

TRACE-LEVEL FLOW (흐름 중심 관점)

상세 관찰 및 트레이스 추적

LLM 호출의 '블랙박스' 부분을 관찰 가능한 구조로 추적한다

- **Telemetry Visibility:** Micrometer + OTEL Span으로 모듈별 실시간 소요 시간(레이턴시) 관찰
- **Exact Token Map:** Langfuse를 통해 질문과 응답 단위별 토큰 비용 매핑
- **Prompt Audit:** Langfuse로 LLM에 전달된 최종 프롬프트(검색 컨텍스트 + 사용자 질문)를 트레이스 단위로 기록·조회

결과: "비용·품질·리스크·흐름을 정량 계측하여 통제 가능"

오늘 정리 & 다음 수업 안내

TODAY SUMMARY

스택 아키텍처 요점 정리

- **Spring AI 역할 범위:** LLM 호출과 외부 컴포넌트(Advisor, Tool, RAG)를 애플리케이션 안의 표준 결합 상태로 변형하는 중심 접착 구조
- **관측 인프라의 본질:** 성공 응답 확인에 가두지 않고, 실행 중간 과정의 흐름(Trace/Span)과 시스템 내부 지표를 가감 없이 출력
- **Langfuse 관찰 포털:** LLM 호출의 토큰 사용량·비용과 프롬프트 이력을 실시간으로 시각화하는 창

#spring_ai #observability #otel_tracing #langfuse_dashboard

UP NEXT LECTURE

다음 수업 미리보기

기술 구성 전체의 지도가 완벽하게 확보되었습니다. 다음 수업부터는 이 지도를 들고, 실무 기준 사례인 Incident Responder 애플리케이션 코드 속으로 들어갑니다.

LECTURE 5 TOPIC

Incident Responder 기준 사례 소개 샘플 코드 흐름 살펴보기

Move to Next Lecture